

## # Informe final - Proyecto Integrador de Telemática

> Reemplazar los campos entre corchetes antes de exportar a PDF. El archivo PDF debe llamarse con el primer apellido de los integrantes, por ejemplo `Perez-Gomez-Ramirez.pdf`.

### ## Portada

**\*\*Proyecto:\*\*** Despliegue de servicio telemático seguro en AWS  
**\*\*Materia:\*\*** Internet: Arquitectura y Protocolos / Telemática  
**\*\*Institucion:\*\*** Universidad EAFIT  
**\*\*Integrantes:\*\*** [Nombre Apellido], [Nombre Apellido], [Nombre Apellido]  
**\*\*Profesor:\*\*** [Nombre del profesor]  
**\*\*Fecha:\*\*** [Fecha de entrega]  
**\*\*Repositorio GitHub:\*\*** [URL del repositorio]  
**\*\*URL publica:\*\*** `https://www.[dominio]`

### ## Objetivo

Desplegar una aplicación web segura en la nube que permita registrar usuarios interesados en estudiar una carrera, almacenar la información en una base de datos, balancear tráfico entre dos servidores web y generar estadísticas con gráficas enviadas por correo a `ialondonoo@eafit.edu.co`.

### ## Descripción del problema

El proyecto exige implementar una infraestructura cloud con acceso público por HTTPS, dominio y DNS, balanceador de carga, dos servidores web, base de datos, Docker y documentación técnica. La aplicación debe registrar nombre, comuna, fecha de ingreso y carrera de interés entre Medicina, Ingeniería, Abogacía y Licenciatura. También debe demostrar balanceo round robin entre dos servidores y generar estadísticas acumuladas.

### ## Requisitos extraídos del enunciado

#### #### Componentes obligatorios

- Cliente externo fuera de la nube.
- Dominio público con registro DNS tipo A.
- HTTPS con certificado de sitio.
- NGINX como proxy inverso y balanceador round robin.
- Dos servidores web en Docker.
- Base de datos en Docker.
- Aplicación web en Python o C.
- Formulario de registro.
- Estadísticas con gráficas.
- Envío de reporte por correo.
- Despliegue final en AWS.

#### #### Restricciones

- El cliente no se despliega en AWS.
- La base de datos no se expone a Internet.
- Los servidores web solo reciben tráfico desde el balanceador.
- El direccionamiento debe coincidir con la VPC/subred real configurada.
- Docker es obligatorio.

#### #### Entregables

- Repositorio GitHub.
- Aplicación funcionando en AWS.
- URL pública HTTPS.
- Informe final en PDF.
- Capturas de configuración y pruebas.

### ## Arquitectura propuesta

La arquitectura se divide en cliente externo, capa pública de balanceo, capa privada de

aplicaciones y capa privada de datos.

```
```text
```

Cliente externo

- > DNS publico
- > EC2 publica con NGINX HTTPS
- > Web Server 1 en ingles
- > Web Server 2 en espanol
- > PostgreSQL
- > Stats + SMTP

```
```
```

```
```mermaid
```

flowchart LR

```
C[Cliente externo] -->|HTTPS 443| DNS[DNS publico]
DNS --> LB[NGINX Load Balancer]
LB -->|Round robin| W1[Web Server 1 - English]
LB -->|Round robin| W2[Web Server 2 - Espanol]
LB -->|/admin/stats| S[Stats]
W1 --> DB[(PostgreSQL)]
W2 --> DB
S --> DB
S -->|SMTP| M[jalondonoo@eafit.edu.co]
```

```
```
```

## Diagrama de red

VPC propuesta:

- VPC: `10.0.0.0/16`
- Subred publica: `10.0.1.0/24`
- Subred privada: `10.0.2.0/24`
- Internet Gateway conectado a la subred publica.
- NAT Gateway opcional para actualizaciones desde subred privada.

## Tabla de direccionamiento

| Componente      | Tipo          | IP privada  | IP publica     | Subred        | Mascara         | Gateway | Puertos                                 |
|-----------------|---------------|-------------|----------------|---------------|-----------------|---------|-----------------------------------------|
| Cliente externo | Cliente       | No aplica   | IP del ISP     | Internet      | No aplica       | ISP     | 443/80 salida                           |
| NGINX LB        | Balanceador   | `10.0.1.10` | `[Elastic IP]` | `10.0.1.0/24` | `255.255.255.0` |         | `10.0.1.1`   80, 443                    |
| Web Server 1    | Web           | `10.0.2.11` | Ninguna        | `10.0.2.0/24` | `255.255.255.0` |         | `10.0.2.1`   5000 desde LB              |
| Web Server 2    | Web           | `10.0.2.12` | Ninguna        | `10.0.2.0/24` | `255.255.255.0` |         | `10.0.2.1`   5000 desde LB              |
| Stats           | Estadisticas  | `10.0.2.30` | Ninguna        | `10.0.2.0/24` | `255.255.255.0` |         | `10.0.2.1`   5000 desde LB, SMTP salida |
| PostgreSQL      | Base de datos | `10.0.2.20` | Ninguna        | `10.0.2.0/24` | `255.255.255.0` |         | `10.0.2.1`   5432 interno               |

Las direcciones son una propuesta. En la entrega final se deben reemplazar por las IPs reales asignadas por AWS.

## Configuración AWS

Se crea una VPC con una subred publica y una privada. El balanceador se ubica en la subred publica con Elastic IP. Los servidores web, stats y base de datos se ubican en la subred privada. Las reglas de seguridad permiten trafico publico solo a `80` y `443` del balanceador.

Security Groups:

- `sg-lb`: entrada `80/443` desde Internet y SSH administrativo restringido.
- `sg-web`: entrada `5000` solo desde `sg-lb`.
- `sg-stats`: entrada `5000` solo desde `sg-lb`, salida SMTP.

- `sg-db`: entrada `5432` solo desde web y stats.

## ## Configuración Docker

Servicios:

- `nginx-lb`: NGINX con puertos publicados `80` y `443`.
- `web-en`: aplicación Flask en inglés.
- `web-es`: aplicación Flask en español.
- `db`: PostgreSQL con volumen persistente.
- `stats`: dashboard de estadísticas y correo.

Comando:

```
```bash
docker compose up -d --build
```
```

## ## Configuración NGINX

NGINX recibe HTTPS y reenvía a los servidores internos:

```
```nginx
upstream backend_web {
    server web-en:5000;
    server web-es:5000;
}
```
```

La política round robin es la política por defecto de NGINX. Los encabezados reenviados son:

- `Host`
- `X-Real-IP`
- `X-Forwarded-For`
- `X-Forwarded-Proto`

## ## Configuración DNS

Registro requerido:

| Tipo | Nombre | Valor                      |
|------|--------|----------------------------|
| A    | `www`  | IP pública del balanceador |

Proveedor recomendado para subdominio gratuito: DuckDNS o FreeDNS. Alternativa: No-IP o dominio institucional.

Comprobación:

```
```bash
nslookup www.[dominio]
```
```

## ## Certificado HTTPS

Opción recomendada: Let's Encrypt con Certbot.

```
```bash
sudo certbot certonly --standalone -d www.[dominio] -d [dominio]
sudo cp /etc/letsencrypt/live/www.[dominio]/fullchain.pem nginx/certs/fullchain.pem
sudo cp /etc/letsencrypt/live/www.[dominio]/privkey.pem nginx/certs/privkey.pem
docker compose restart nginx-lb
```
```

Opción académica: certificado autofirmado para pruebas. Esta opción genera advertencia en el navegador y no debe usarse si se exige certificado público válido.

## ## Aplicacion Web Server 1 - Ingles

Ruta: `/` cuando NGINX redirige al contenedor `web-en`.

Campos:

- Name.
- Commune zone: Commune 1 a Commune 10.
- Date/time of entry.
- Undergraduate program: Medicine, Engineering, Law, Bachelor / Education.

Indicador:

```
```text
Served by Web Server 1 - English
```
```

## ## Aplicacion Web Server 2 - Espanol

Ruta: `/` cuando NGINX redirige al contenedor `web-es`.

Campos:

- Nombre.
- Zona de comuna: Comuna 1 a Comuna 10.
- Fecha de ingreso.
- Carrera: Medicina, Ingeniería, Abogacía, Licenciatura.

Indicador:

```
```text
Atendido por Web Server 2 - Espanol
```
```

## ## Base de datos

Motor: PostgreSQL.

Tabla:

```
```sql
CREATE TABLE registrations (
  id SERIAL PRIMARY KEY,
  name VARCHAR(120) NOT NULL,
  commune VARCHAR(20) NOT NULL,
  program VARCHAR(50) NOT NULL,
  language VARCHAR(20) NOT NULL,
  entry_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  served_by VARCHAR(50),
  client_ip VARCHAR(80)
);
```
```

Comprobacion:

```
```bash
docker compose exec db psql -U telematica_user -d telematica \
-c "SELECT * FROM registrations ORDER BY id DESC;"
```
```

## ## Aplicacion de estadisticas

URL:

```
```text
https://www.[dominio]/admin/stats
```
```

Funciones:

- Total de usuarios registrados.
- Total de usuarios por comuna.
- Total de usuarios por carrera.
- Total de usuarios por comuna y carrera.
- Graficas generadas con Matplotlib.
- Envio de reporte por SMTP.

Variables:

```
```env
SMTP_HOST=
SMTP_PORT=587
SMTP_USER=
SMTP_PASSWORD=
SMTP_FROM=
SMTP_TO=ialondonoo@eafit.edu.co
```
```

## Pruebas realizadas

### DNS

```
```bash
nslookup www.[dominio]
```
```

Resultado: [captura y descripcion].

### HTTPS

```
```bash
curl -lv https://www.[dominio]
```
```

Resultado: [captura y descripcion].

### Round robin

```
```bash
curl -k https://www.[dominio]
curl -k https://www.[dominio]
curl -k https://www.[dominio]
```
```

Resultado: se observa alternancia entre servidor en ingles y servidor en espanol.

### Registro

Resultado: [captura de formulario y confirmacion].

### Base de datos

Resultado: [captura de consulta SQL].

### Estadísticas

Resultado: [captura del dashboard].

### Correo

Resultado: [captura del correo enviado a ialondonoo@eafit.edu.co].

### ### Simultaneidad

```
```bash
for i in $(seq 1 20); do curl -k -s https://www.[dominio] >/dev/null & done; wait
```
```

Resultado: [descripcion].

### ## Problemas encontrados y soluciones

| Problema                               | Causa                                           | Solucion                                                   |
|----------------------------------------|-------------------------------------------------|------------------------------------------------------------|
| El navegador muestra advertencia HTTPS | Certificado autofirmado                         | Emitir certificado con Let's Encrypt para el dominio real  |
| DNS no resuelve                        | Registro A incorrecto o propagacion pendiente   | Verificar IP publica y esperar propagacion                 |
| No alterna round robin                 | Cache, una app caida o upstream mal configurado | Revisar `docker compose ps` y logs de NGINX                |
| DB rechaza conexion                    | Variables `.env` o Security Group incorrectos   | Validar `DB_HOST`, usuario, password y puerto 5432 interno |
| Correo no sale                         | SMTP sin credenciales o puerto bloqueado        | Configurar `SMTP_*`, app password y salida 587/465         |

### ## Conclusiones

El proyecto integra conceptos de telematica, redes, protocolos, contenedores y nube. La solucion implementa acceso seguro por HTTPS, DNS publico, balanceo con NGINX, aplicaciones web en Docker, persistencia en PostgreSQL, estadisticas y envio de correo. La separacion entre cliente externo, capa publica y servicios internos permite demostrar una arquitectura coherente y segura para un servicio web desplegado en AWS.

### ## Anexos de comandos

```
```bash
cp .env.example .env
./scripts/deploy.sh
docker compose ps
docker compose logs -f nginx-lb
curl -k https://www.[dominio]
curl -k https://www.[dominio]/admin/stats
./scripts/backup-db.sh
./scripts/test.sh https://www.[dominio]
```
```

### ## Fuentes consultadas

- AWS VPC Internet Gateway: [https://docs.aws.amazon.com/vpc/latest/userguide/VPC\\_Internet\\_Gateway.html](https://docs.aws.amazon.com/vpc/latest/userguide/VPC_Internet_Gateway.html)
- Certbot NGINX/Linux: <https://certbot.eff.org/instructions?ws=nginx&os=linux>
- DuckDNS: <https://www.duckdns.org/>
- No-IP Free Dynamic DNS: <https://www.noip.com/free>
- FreeDNS afraid.org: <https://freedns.afraid.org/>